61.501 Production Machine Learning — Homework 1 (Questions 1–5)

This notebook contains the implementations, numerical results, plots, and written explanations for the assignment.

The regression sections use the squared-loss empirical risk $R(\theta) = \frac{1}{2n}\sum_i(\hat{y}_i - y_i)^2$.

In [...

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import clear_output

# Search upward from the current directory so the notebook works from the project or its subfolders.
candidates = [Path.cwd(), *Path.cwd().parents]
ROOT = next(p for p in candidates if (p / '1/train_1_5.csv').exists())


def empirical_risk(X, y, theta):
    """Compute squared-loss empirical risk: sum((prediction - target)^2) / (2n)."""
    return np.sum((X @ theta - y) ** 2) / (2 * len(y))


def design_matrix(x, degree=1):
    """Build polynomial features [x, x^2, ..., x^degree, 1], including an intercept."""
    return np.column_stack([x ** k for k in range(1, degree + 1)] + [np.ones_like(x)])


def PolyRegress(x, y, d):
    """Fit a degree-d polynomial by least squares; return weights and training risk."""
    X_poly = design_matrix(x, d)
    theta_poly = np.linalg.lstsq(X_poly, y, rcond=None)[0]
    return theta_poly, empirical_risk(X_poly, y, theta_poly)

# Clear stale execution output so only English assignment content remains in the saved notebook.
clear_output(wait=True)
```

Q1. Linear Classification — Perceptron

Q1(a) One epoch and Q1(b) five epochs

Labels are $y \in \{-1, +1\}$. Initialize $\theta = 0$ and $b = 0$, then process the training examples in their original order. If $y(\theta^T x + b) \leq 0$, update $\theta \leftarrow \theta + yx$ and $b \leftarrow b + y$. Run the algorithm for one and five epochs without shuffling the data.

Q1(c) Executable implementation

The code cell below implements the algorithm and reports the learned parameters and test accuracy for both epoch settings.

In [...

```
train = np.loadtxt(ROOT / '1/train_1_5.csv', delimiter=',')
test = np.loadtxt(ROOT / '1/test_1_5.csv', delimiter=',')


def perceptron(train, test, epochs):
    """Run sequential perceptron updates with zero-initialized weights and offset."""
    theta = np.zeros(train.shape[1] - 1)
    offset = 0.0
    for _ in range(epochs):
        for row in train:  # Keep the original order; the assignment forbids shuffling.
            x, y = row[:-1], row[-1]
            # A non-positive signed score means misclassification or a point on the boundary.
            if y * (theta @ x + offset) <= 0:
                theta += y * x
                offset += y
    pred = np.where(test[:, :-1] @ theta + offset >= 0, 1, -1)
    return theta, offset, np.mean(pred == test[:, -1])


for epochs in (1, 5):
    theta, offset, accuracy = perceptron(train, test, epochs)
    print(f'{epochs} epoch(s): theta={theta}, offset={offset:.1f}, test accuracy={accuracy:.4%}')
```

```
1 epoch(s): theta=[-2.4483233  -5.88190688], offset=0.0, test accuracy=96.7066%
5 epoch(s): theta=[-2.05895878 -8.83521606], offset=1.0, test accuracy=96.7066%
```

Q2. Linear and Polynomial Regression

Q2(a) Closed-form linear regression

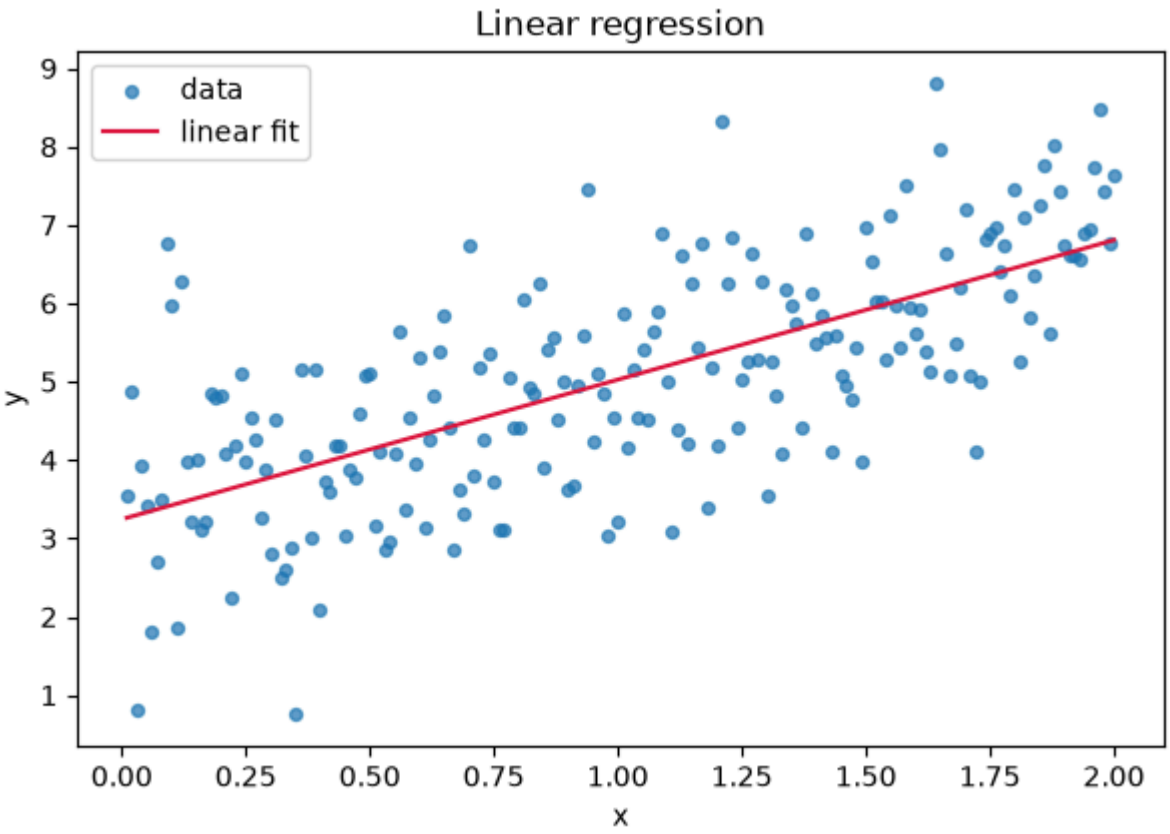
The final column of the design matrix is the intercept term. The least-squares solution is computed with a numerically stable solver rather than by explicitly inverting a matrix. The empirical risk is also reported.

In [...

```
x = np.loadtxt(ROOT / '2/hw1x.dat')
y = np.loadtxt(ROOT / '2/hw1y.dat')
X = design_matrix(x)
theta_linear = np.linalg.lstsq(X, y, rcond=None)[0]
print('2(a) theta [slope, intercept]:', theta_linear)
print('2(a) empirical risk:', empirical_risk(X, y, theta_linear))

grid = np.linspace(x.min(), x.max(), 500)
plt.figure(figsize=(7, 4.5))
plt.scatter(x, y, s=18, alpha=.7, label='data')
plt.plot(grid, theta_linear[0] * grid + theta_linear[1], color='crimson', label='linear fit')
plt.xlabel('x'); plt.ylabel('y'); plt.title('Linear regression'); plt.legend(); plt.show()
```

```
2(a) theta [slope, intercept]: [1.78157138 3.2447227 ]
2(a) empirical risk: 0.5812605752543937
```



Q2(b) Batch Gradient Descent and Stochastic Gradient Descent

For squared loss $R(\theta) = \frac{1}{2n}\sum_i (x_i^T\theta - y_i)^2$, the gradient is $X^T(X\theta - y)/n$. Batch GD updates once per epoch using all examples; SGD updates after each example in the original data order. The learning rate is $\eta = 0.01$, and each method runs for five epochs.

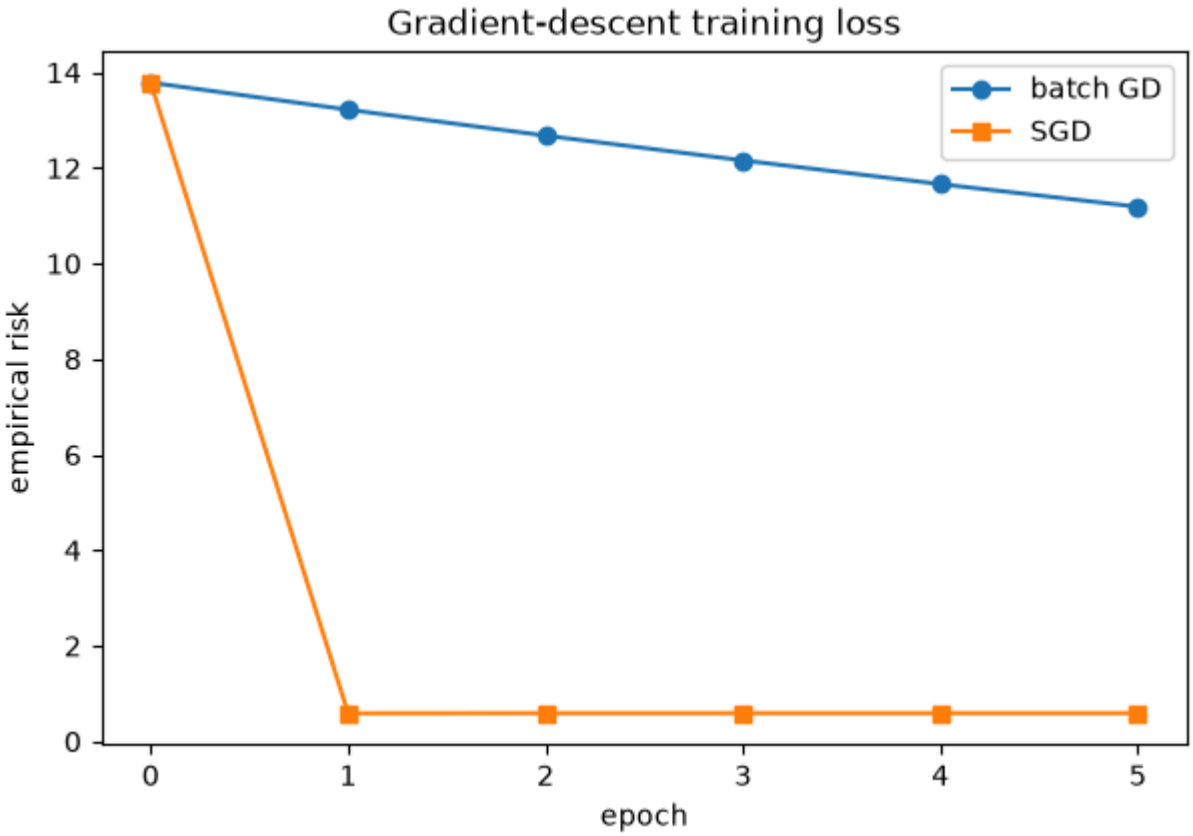
In [...

```
eta, n = 0.01, len(y)
# Batch GD uses the average gradient over all samples and updates once per epoch.
theta_gd = np.zeros(X.shape[1])
gd_risk = [empirical_risk(X, y, theta_gd)]
for _ in range(5):
    gradient = X.T @ (X @ theta_gd - y) / n
    theta_gd -= eta * gradient
    gd_risk.append(empirical_risk(X, y, theta_gd))
gd_best_epoch = int(np.argmin(gd_risk))
print(f'Q2(b) Batch GD: theta={theta_gd}; minimum risk={gd_risk[gd_best_epoch]:.8f} at epoch {gd_best_epoch}')

# SGD updates after each sample, preserving the order in the data file.
theta_sgd = np.zeros(X.shape[1])
sgd_risk = [empirical_risk(X, y, theta_sgd)]
sgd_best = (sgd_risk[0], 0, theta_sgd.copy())
for epoch in range(1, 6):
    for xi, yi in zip(X, y):
        sample_gradient = (xi @ theta_sgd - yi) * xi
        theta_sgd -= eta * sample_gradient
    current = empirical_risk(X, y, theta_sgd)
    sgd_risk.append(current)
    if current < sgd_best[0]:
        sgd_best = (current, epoch, theta_sgd.copy())
print(f'Q2(b) SGD: theta at minimum={sgd_best[2]}; minimum risk={sgd_best[0]:.8f} at epoch {sgd_best[1]}')

plt.figure(figsize=(7, 4.5))
plt.plot(range(6), gd_risk, 'o-', label='batch GD')
plt.plot(range(6), sgd_risk, 's-', label='SGD')
plt.xlabel('epoch'); plt.ylabel('empirical risk'); plt.title('Gradient-descent training loss'); plt.legend(); plt.sh
ow()
```

```
Q2(b) Batch GD: theta=[0.27032973 0.24127422]; minimum risk=11.18724893 at epoch 5
Q2(b) SGD: theta at minimum=[1.94140961 3.09712867]; minimum risk=0.58560358 at epoch 1
```



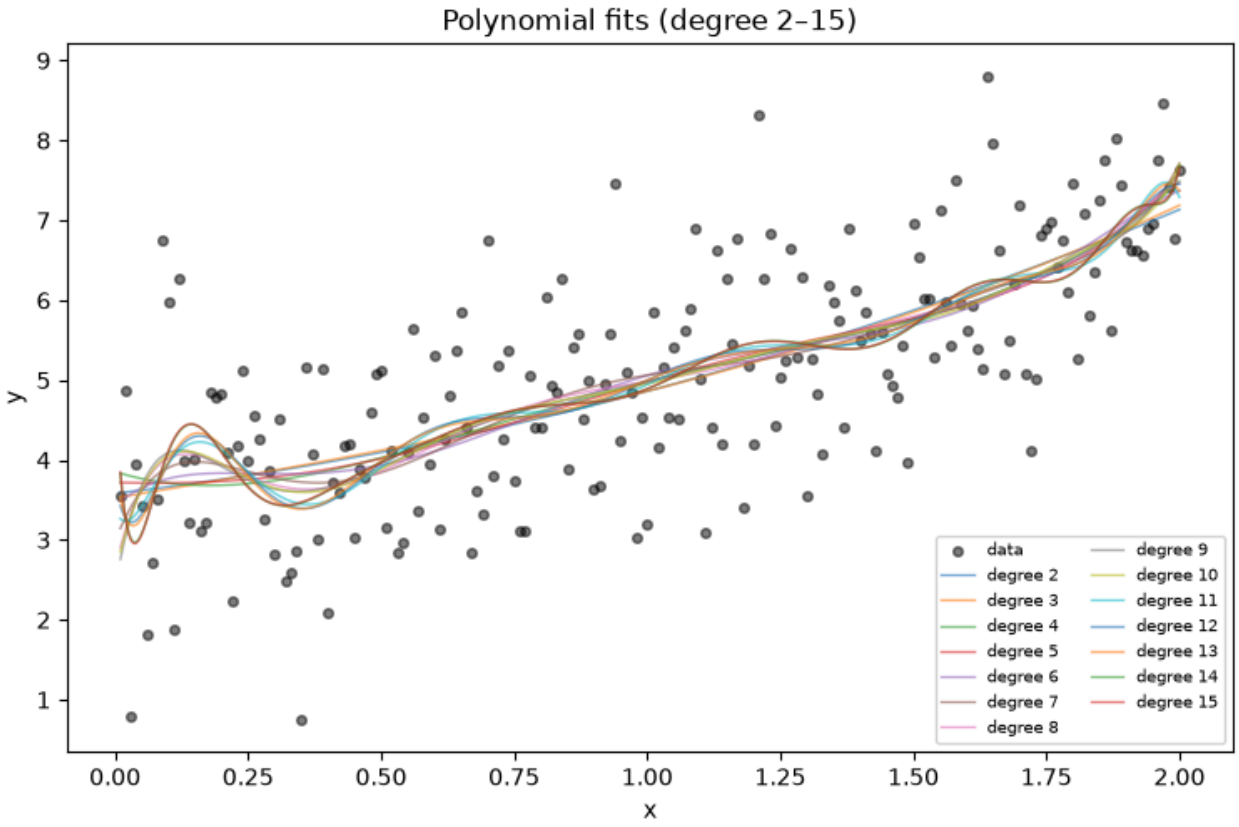
Q2(c) Polynomial regression: degrees 2 through 15

`PolyRegress(x, y, d)` augments the features to $[x, x^2, \ldots, x^d, 1]$, performs closed-form least squares, and returns the weights and training empirical risk. The code plots the quadratic fit and fits of degrees 3–15. For nested models, the optimal training error should not increase; validation data is needed to assess generalization.

In […

```
poly_risks = {}
poly_thetas = {}
plt.figure(figsize=(9, 5.5))
plt.scatter(x, y, s=16, alpha=.5, color='black', label='data')
for degree in range(2, 16):
    # PolyRegress expands features, fits the model, and returns its empirical risk.
    poly_thetas[degree], poly_risks[degree] = PolyRegress(x, y, degree)
    plt.plot(grid, design_matrix(grid, degree) @ poly_thetas[degree],
             alpha=.6, linewidth=1, label=f'degree {degree}')
    print(f'Q2(c) degree {degree:2d}: empirical risk={poly_risks[degree]:.8f}')
plt.xlabel('x'); plt.ylabel('y'); plt.title('Polynomial fits (degree 2–15)')
plt.legend(ncol=2, fontsize=7); plt.show()
worse = [d for d in range(3, 16) if poly_risks[d] > poly_risks[d - 1] + 1e-10]
print('First degree at which training error worsens:', worse[0] if worse else 'none through degree 15')
```

```
Q2(c) degree  2: empirical risk=0.57030161
Q2(c) degree  3: empirical risk=0.57008444
Q2(c) degree  4: empirical risk=0.56461639
Q2(c) degree  5: empirical risk=0.56389682
Q2(c) degree  6: empirical risk=0.56134330
Q2(c) degree  7: empirical risk=0.55623525
Q2(c) degree  8: empirical risk=0.55378058
Q2(c) degree  9: empirical risk=0.55292311
Q2(c) degree 10: empirical risk=0.55259018
Q2(c) degree 11: empirical risk=0.54535163
Q2(c) degree 12: empirical risk=0.54417449
Q2(c) degree 13: empirical risk=0.54384051
Q2(c) degree 14: empirical risk=0.53901577
Q2(c) degree 15: empirical risk=0.53899419
```




```
First degree at which training error worsens: none through degree 15
```

Q3. Ridge Regression

Q3(a) Closed-form ridge solution

As instructed, the first 10 examples form the validation set and the remaining 40 form the training set. The implementation uses the lecture-note formula $\hat{\theta} = (n\lambda I + X^T X)^{-1} X^T y$, with $n = 40$, and reports the solution for $\lambda = 0.15$.

Q3(b) Selecting the regularization parameter

Training and validation empirical risks are computed on the prescribed logarithmic grid. The loss curves are plotted, and the value of λ that minimizes validation loss is reported.

In […

```
ridge_X = np.loadtxt(ROOT / '3/hw1_ridge_x.dat', delimiter=',')
ridge_y = np.loadtxt(ROOT / '3/hw1_ridge_y.dat')
```

```python
# The assignment specifies the first 10 examples for validation and the remaining 40 for training.
vX, vY = ridge_X[:10], ridge_y[:10]
tX, tY = ridge_X[10:], ridge_y[10:]
n_train = len(tY)

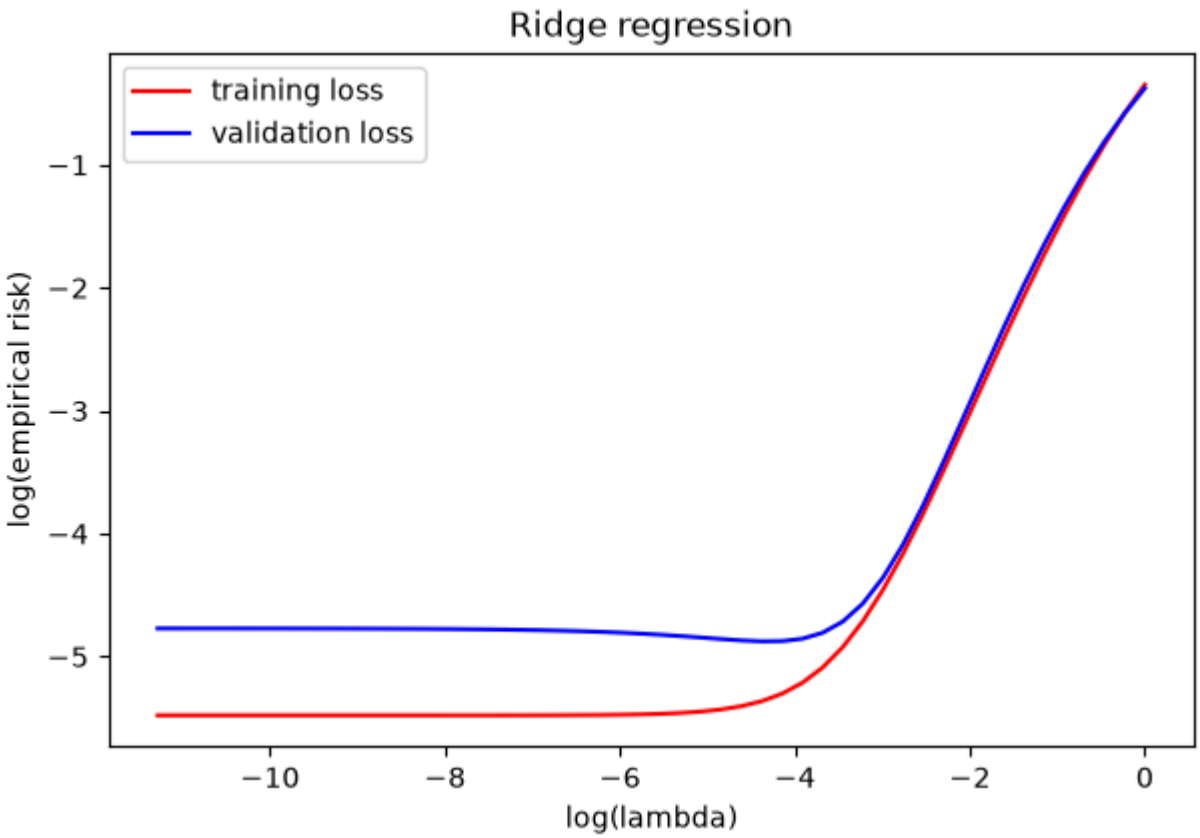

def ridge_regression(tX, tY, lam):
    """Return the closed-form ridge solution specified in the assignment."""
    n = len(tY)
    regularized_gram = tX.T @ tX + n * lam * np.eye(tX.shape[1])
    return np.linalg.solve(regularized_gram, tX.T @ tY)


# Q3(a): compute the weights and train/validation risks at the specified lambda.
theta_015 = ridge_regression(tX, tY, 0.15)
print('Q3(a) theta at lambda=0.15:', theta_015)
print('train risk:', empirical_risk(tX, tY, theta_015))
print('validation risk:', empirical_risk(vX, vY, theta_015))

# Q3(b): select the lambda with the lowest validation risk on the prescribed grid.
lambdas = 10.0 ** (-np.arange(0, 5, 0.1))
train_loss = np.array([empirical_risk(tX, tY, ridge_regression(tX, tY, lam)) for lam in lambdas])
val_loss = np.array([empirical_risk(vX, vY, ridge_regression(tX, tY, lam)) for lam in lambdas])
best = int(np.argmin(val_loss))
print(f'Q3(b) best lambda={lambdas[best]:.8g}; validation risk={val_loss[best]:.8f}; training risk={train_loss[bes
t]:.8f}')

plt.figure(figsize=(7, 4.5))
plt.plot(np.log(lambdas), np.log(train_loss), 'r', label='training loss')
plt.plot(np.log(lambdas), np.log(val_loss), 'b', label='validation loss')
plt.xlabel('log(lambda)'); plt.ylabel('log(empirical risk)'); plt.title('Ridge regression'); plt.legend(); plt.show
()
```

```
Q3(a) theta at lambda=0.15: [-0.57939825  1.15033028  0.04934122 -1.59867896]
train risk: 0.05738711689975613
validation risk: 0.06260197726334511
Q3(b) best lambda=0.012589254; validation risk=0.00762781; training risk=0.00469152
```



Q4. Support Vector Machines

Q4(a) Maximum-margin separator

A linear classifier uses the hyperplane $w^T x + b = 0$ as its decision boundary. Among hyperplanes that separate the training examples, an SVM selects one that maximizes the distance to the nearest examples. Under canonical scaling, the margin width is $2/\|w\|$, so maximizing the margin is equivalent to minimizing $\frac{1}{2}\|w\|^2$. A large margin is generally more robust to small perturbations. Minimizing the margin instead would produce a narrow, fragile boundary that is sensitive to noise and can generalize poorly.

Q4(b) Soft margin and the parameter C

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(w^T\phi(x_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

Slack variables ξ_i quantify margin violations. A small C tolerates more violations in exchange for a wider margin; a large C emphasizes training-set correctness and may overfit. Select C using a validation set or cross-validation.

Q4(c) Kernel construction and verification

$K_1(x, x') = x^T x'$ is the linear kernel, and $K_2(x, x') = (x^T x')^2$ is a valid degree-two kernel. The sum and product of valid kernels are valid, so:

$$K_+(x, x') = K_1 + K_2 = x^T x' + (x^T x')^2, \qquad K_\times(x, x') = K_1 K_2 = (x^T x')^3.$$

The proposed $K(x, x') = \|x\|^2 + \|x'\|^2 - 2x^T x' = \|x - x'\|^2$ is not a valid kernel in general. For one-dimensional inputs 0 and 1, its Gram matrix is $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, with eigenvalues 1 and -1. Since it is not positive semidefinite, the function fails the kernel condition.

Q5. Logistic Regression

Q5(a) Interpreting the predicted probability

Logistic regression outputs $h_\theta(x) = P(y = 1 \mid x; \theta)$. If $h_\theta(x) = 0.65$, then $P(y = 1 \mid x; \theta) = 0.65$ and $P(y = 0 \mid x; \theta) = 1 - 0.65 = 0.35$. Therefore, the correct choices are **1 and 4**.

Q5(b) Implementing logistic regression from scratch

The dataset contains 455 training examples and 114 test examples. Use the two specified features, standardize them using training-set statistics only, and add an intercept. Minimize binary cross-entropy with a hand-written batch gradient descent implementation. Report test accuracy and plot the training-loss curve and decision boundary.

> This notebook is a study and reproducibility aid. Review the course academic-integrity rules, understand the implementation, and prepare your own final submission.

In […

```
import pandas as pd

train5 = pd.read_csv(ROOT / '5/train.csv')
test5 = pd.read_csv(ROOT / '5/test.csv')
feature_names = ['mean_radius', 'mean_texture']
X_train_raw = train5[feature_names].to_numpy(dtype=float)
y_train = train5['label'].to_numpy(dtype=float)
X_test_raw = test5[feature_names].to_numpy(dtype=float)
y_test = test5['label'].to_numpy(dtype=int)

# Fit scaling statistics on the training set only to prevent test-data leakage.
feature_mean = X_train_raw.mean(axis=0)
feature_std = X_train_raw.std(axis=0)
X_train_scaled = (X_train_raw - feature_mean) / feature_std
X_test_scaled = (X_test_raw - feature_mean) / feature_std
X_train_logistic = np.column_stack([np.ones(len(X_train_scaled)), X_train_scaled])
X_test_logistic = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])


def sigmoid(z):
    """Map logits to probabilities; clipping prevents exponential overflow."""
    z = np.clip(z, -40, 40)
    return 1 / (1 + np.exp(-z))


def binary_cross_entropy(X, labels, weights):
    """Compute the mean binary cross-entropy using a numerically stable formula."""
    logits = X @ weights
    return float(np.mean(np.logaddexp(0, logits) - labels * logits))

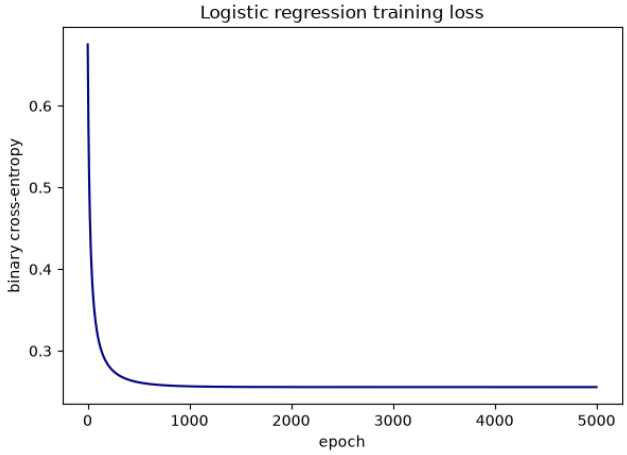

weights = np.zeros(X_train_logistic.shape[1])
learning_rate = 0.1
epochs = 5000
train_losses = []
for _ in range(epochs):
    probabilities = sigmoid(X_train_logistic @ weights)
    # For logistic cross-entropy, the batch gradient is X.T @ (p - y) / n.
    gradient = X_train_logistic.T @ (probabilities - y_train) / len(y_train)
    weights -= learning_rate * gradient
    train_losses.append(binary_cross_entropy(X_train_logistic, y_train, weights))

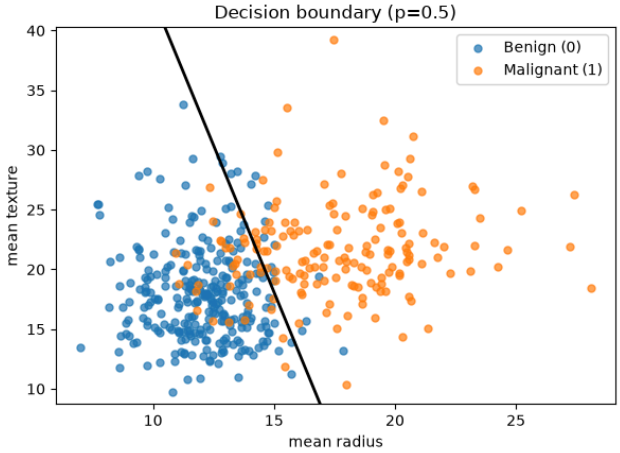
test_probabilities = sigmoid(X_test_logistic @ weights)
test_predictions = (test_probabilities >= 0.5).astype(int)
test_accuracy = np.mean(test_predictions == y_test)
print(f'Train samples: {len(y_train)}, test samples: {len(y_test)}')
print('weights [intercept, standardized radius, standardized texture]:', weights)
print(f'Final training cross-entropy: {train_losses[-1]:.6f}')
print(f'Test accuracy: {test_accuracy:.4%}')

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].plot(np.arange(1, epochs + 1), train_losses, color='navy')
axes[0].set(xlabel='epoch', ylabel='binary cross-entropy', title='Logistic regression training loss')

radius_grid = np.linspace(X_train_raw[:, 0].min() - 1, X_train_raw[:, 0].max() + 1, 250)
texture_grid = np.linspace(X_train_raw[:, 1].min() - 1, X_train_raw[:, 1].max() + 1, 250)
RR, TT = np.meshgrid(radius_grid, texture_grid)
raw_grid = np.column_stack([RR.ravel(), TT.ravel()])
scaled_grid = (raw_grid - feature_mean) / feature_std
logits_grid = np.column_stack([np.ones(len(scaled_grid)), scaled_grid]) @ weights
probability_grid = sigmoid(logits_grid).reshape(RR.shape)
for label, name, color in [(0, 'Benign (0)', 'tab:blue'), (1, 'Malignant (1)', 'tab:orange')]:
    mask = y_train.astype(int) == label
    axes[1].scatter(X_train_raw[mask, 0], X_train_raw[mask, 1], s=25, alpha=.7, label=name, color=color)
# The decision boundary is the contour where the estimated positive-class probability equals 0.5.
axes[1].contour(RR, TT, probability_grid, levels=[0.5], colors='black', linewidths=2)
axes[1].set(xlabel='mean radius', ylabel='mean texture', title='Decision boundary (p=0.5)')
axes[1].legend()
plt.tight_layout()
plt.show()
```

```
Train samples: 455, test samples: 114
weights [intercept, standardized radius, standardized texture]: [-0.66658154  3.6757874   0.88873616]
Final training cross-entropy: 0.255654
Test accuracy: 89.4737%
```


Logistic regression training loss
binary cross-entropy
epoch
0.6
0.5
0.4
0.3
0
1000
2000
3000
4000
5000


Decision boundary (p=0.5)
Benign (0)
Malignant (1)
mean texture
mean radius
40
35
30
25
20
15
10
10
15
20
25